

El loop de desarrollo, paso a paso

Un ciclo de desarrollo con agentes en el que la máquina despacha y verifica, y el humano decide exactamente **tres veces**: al congelar el spec, al promover un slice a la cola, y al mergear. Todo lo demás — el parseo del spec, la creación de issues, el aislamiento en worktrees, el claim, el release, la detección de residuo — lo hace el loop. Esta página documenta cada paso y el formato exacto de cada artefacto que viaja entre ellos.

PLUGIN 0.29.0	CONTRATO DE SLICES v16	PLANTILLA DEL SPEC v2	SKILLS FORKADOS 11 · sp 6.0.3
COMANDOS 4	PUERTAS HUMANAS 3		

01 / EL MAPA

Tres carriles, cuatro fases, tres puertas

Hay **dos sesiones vivas por repo con papeles opuestos**: la coordinadora, en el checkout principal, que groomea, despacha, revisa y mergea; y una sesión despachada por slice, en su propio worktree, que implementa y para. El humano no está en un carril de ejecución: está en las juntas.

● HUMANO	decide, no ejecuta	● COORDINADORA	checkout principal
● AGENTE DEL SLICE	.worktrees/<n>	● HOOK	automático, sin sesión

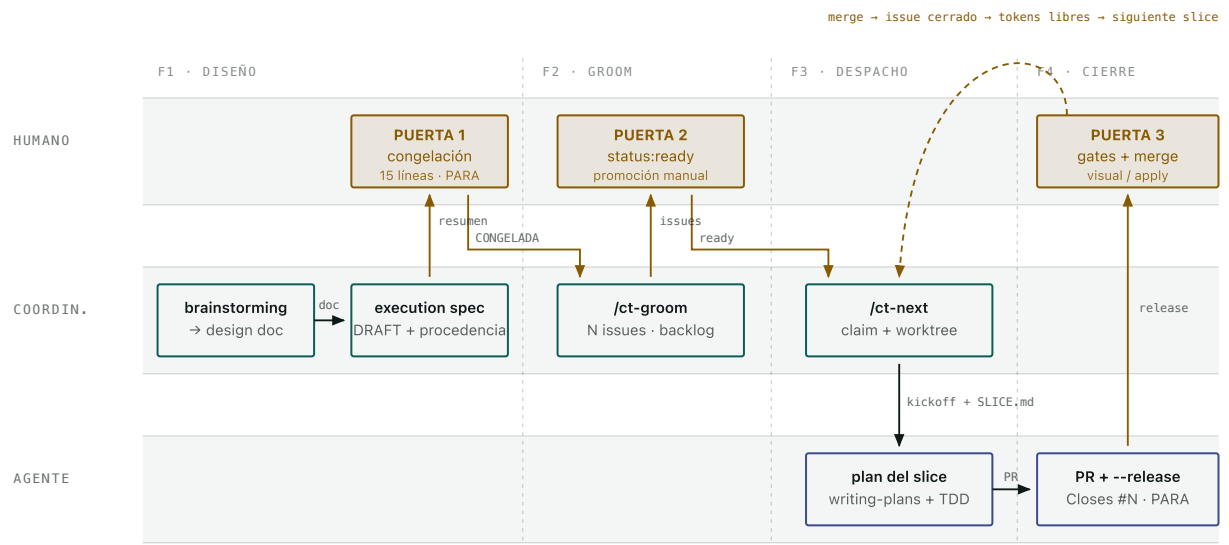


Fig. 1 — El ciclo completo de un epic. Las flechas ámbar cruzan una junta humana; las neutras son automáticas. El arco discontinuo de arriba es el único cierre del bucle: hasta que un PR se mergea y su issue se cierra como *completed*, ese slice retiene sus tokens de área y ningún vecino que los comparta se despacha.

EL MODELO DE DOS NIVELES

Control Tower es el patrón de desarrollo dirigido por subagentes, un nivel por encima.

Nivel epic: la tabla de slices es el fichero de plan, `/ct-next` es el coordinador, la sesión despachada es el implementador, y la revisión humana del PR es el *two-stage review*. Nivel slice: dentro del worktree corre el conjunto entero de skills forkados — `writing-plans`, `subagent-driven-development`, `test-driven-development`, `systematic-debugging`. El mismo patrón, dos escalas.

Dieciséis pasos, de la idea al merge

Cada paso dice quién lo ejecuta, con qué comando, qué lee y qué escribe. Los pasos con carril ámbar son los que no puede hacer la máquina.

FASE 0 Bootstrap una vez por repo · /ct-init

S0

COORD.

Preparar el repo para el loop

El scaffolder siembra el terreno y **nada más**: no es un planificador. Deja el contrato de la tabla de slices dentro de `AGENTS.md`, delimitado por marcadores y con su propia versión, para que quien escriba specs para ese repo tenga la cabecera exacta a mano. Rellenar los comandos reales del repo (build, test, lint, CI) en `AGENTS.md` sí exige explorar.

```
bash ${CLAUDE_PLUGIN_ROOT}/scripts/ct-init.sh "$(pwd)"
```

Dos avisos que hay que transmitir enteros, no resolver: si el repo ya traía su propio protocolo de claim, su propia ruta de worktrees o su propio fichero de estado, elegir cuál manda es decisión del usuario. Y si dice que *no se ha podido comprobar* (falta `node`), eso no es «no hay ninguna».

ESCRIBE	NO HACE
<code>.agent/STATE.md</code> , <code>AGENTS.md</code> (bloque del contrato), <code>.gitignore</code> (<code>.worktrees/</code> + <code>.agent/SLICE.md</code>)	No rellena <code>next_action</code> — se queda en (sin slice asignado). No registra el repo en el tower.

FASE 1 Diseño y congelación nivel epic · control-tower-loop:brainstorming

S1

COORD.

Explorar y preguntar, una pregunta a la vez

Contexto del proyecto primero (ficheros, docs, commits recientes). Después, **una sola pregunta por mensaje**, preferiblemente de opción múltiple, sobre propósito, restricciones y criterios de éxito. Si el encargo describe varios subsistemas independientes, se señala *antes* de refinar detalles: eso se descompone en sub-proyectos, cada uno con su propio ciclo.

Luego 2–3 enfoques con sus *trade-offs* y una recomendación liderando, no una lista exhaustiva. Y el diseño presentado **por secciones**, pidiendo el visto bueno de cada una.

PUERTA DURA DEL SKILL

Cero código, cero scaffolding, cero skills de implementación hasta que el diseño esté aprobado. También en proyectos «triviales».

S2

• COORD.

Escribir el design doc y auto-revisarlo

Se commitea en `docs/superpowers/specs/YYYY-MM-DD-<tema>-design.md`. Después, cuatro pasadas con ojos frescos: **placeholders** (TBD, TODO, secciones a medias), **consistencia interna**, **alcance** (¿cabe en un plan?) y **ambigüedad** (si algo admite dos lecturas, se elige una y se hace explícita). Se arregla en el sitio; no hay segunda ronda.

ESCRIBE

`...-design.md`, commiteado

DESTINO

Será el Handoff origen del execution spec. Tras la congelación es historia: nadie lo edita.

S3

• COORD.

Escribir el execution spec en DRAFT

Un fichero por epic, desde `_TEMPLATE-execution-spec.md`. No es «el join y nada más»: es **el registro del join y de sus entradas comprimidas**, con un criterio único de qué entra — *lo que el ejecutor necesita y no puede derivar*.

Cada decisión congelada lleva su procedencia: **hablada** (con la cita si se tiene), **deducida** (se sigue de una hablada o de una regla ya escrita del repo) o **propuesta** (la pone el agente). Regla dura: **una propuesta no se congela** — se pregunta, y entonces es hablada, o baja a «Decisiones aparcadas». Los huecos no se rellenan.

ESCRIBE

`...-execution.md`, Estado: DRAFT

ADMITIDO EN DRAFT

[NEEDS CLARIFICATION: ...]

PROHIBIDO AQUÍ

El plan de cada slice. Se escribe después, contra el código real.

S4

• HUMANO

Puerta 1 — la congelación

Esta puerta existe por una frase: «yo no me suelo leer los specs, doy por hecho que lo hablado durante el brainstorming es ok y va al spec». Si el humano no lee el spec, quien lo escribe podría cerrar decisiones con firma ajena. La respuesta son las dos piezas de S3 (procedencia) y esta puerta.

Se presentan como mucho quince líneas, en el chat, que caben en una pantalla: la hipótesis en una o dos líneas, cada decisión congelada a una línea

con su etiqueta de procedencia, y el anti-scope. **Y se para.**

EL OK MUTA DRAFT → CONGELADA + fecha de congelación	SI PIDE CAMBIOS Se corrige el spec y se vuelve a presentar el resumen
SIN OK No hay groom. Es la única lectura del ciclo, por diseño.	

S5

● COORD.

Groom en seco: la puerta de congelación y la validación de la tabla

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-groom.mjs "<spec>" --repo "<ow
```

Antes incluso de mirar la tabla, **dos comprobaciones sobre el spec entero** — dos greps en la pasada que groom ya hacía:

- **[NEEDS CLARIFICATION sin resolver → exit 2.** Se admiten en DRAFT; congelar con un hueco pendiente es inválido. El mensaje nombra cuántos hay y la línea del primero.
- **## Hipótesis ausente o vacía → exit 2.** Sin apuesta falsable no es un epic. El trabajo sin apuesta (mantenimiento, bugfixes) va como issues sueltos. Groom solo mira presencia; la *calidad* la juzga el humano al congelar.

Después, la tabla entera, y **todos los fallos juntos en un único exit 2:** cabecera sin `Slice + Dep`, falta la columna `#`, hueco a mitad del bloque, un `#` que no es entero puro, filas con celdas de más o de menos, `Dep` ilegible o apuntando a un `#` inexistente o a sí misma, un `Gate` fuera del vocabulario, o una fila que pide y renuncia al mismo gate.

El dry-run valida exactamente lo mismo que la corrida real — un dry-run que valida menos es una trampa. Con `--repo` ya no es offline: lee los issues de verdad para poder comparar.

S6

● COORD.

Groom real: un issue por fila

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-groom.mjs "<spec>" --repo "<ow
```

Todo lo que lee va por delante de todo lo que escribe. Validación, resolución y verificación del enlace al spec, enumeración de issues, labels, milestones y —

con `--project` — la introspección del Project v2 (que exige un campo de iteración llamado exactamente `Sprint` y una iteración que cubra hoy). Si aborta en cualquiera de esos pasos, no ha creado nada.

Superada la validación, el orden de escritura es **milestone → labels → issues → alta en el Project con su Sprint**. No hay transacción y no se finge que la haya: un fallo de red ahí deja creado lo anterior. De un abort a mitad se sale **volviendo a correr, no limpiando a mano**: el milestone se reutiliza por título, de las labels solo se crean las que faltan, y los issues se reconocen por su marcador `<!-- ct-order:N -->`.

ALCANCE	IDEMPOTENCIA
Una invocación = un milestone = un epic. Los <code>#</code> son 1..N por epic : cada spec puede empezar en 1.	Solo por existencia. Reportar divergencia ≠ converger: sin <code>--reconcile</code> nunca edita un issue existente (exit 3).

S7

● HUMANO

Puerta 2 — promover a la cola

Groom crea los issues en `status:backlog`, **nunca en `status:ready`**. Es una gate humana deliberada: `/ct-next` no considera despachable nada que no esté en `ready`.

```
gh issue edit <n> --repo <owner/repo> --add-label status:ready --rem
```

Si `/ct-next` responde «No hay slices despachables» justo después de un groom, es casi seguro que sea esto. Al terminar, groom recuerda por `stderr` cuántos issues del epic siguen en backlog — contando también los que ya existían, porque el criterio es el estado real, no «acabo de crear algo».

S8

● COORD.

Dry-run del dispatcher: comprobar lo que el run real necesita

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-next.mjs --repo "<owner/repo>"
```

Verifica que `cmux` esté en el PATH, que el agente se vea desde aquí (solo aviso: quien lo resuelve de verdad es el shell de login), que el `CLAUDE_CONFIG_DIR` resuelto exista, que el worktree y la rama `feat/<n>` estén libres, y que el kickoff se renderice. **Las mismas comprobaciones corren en la corrida real antes de escribir ningún claim.**

Imprime el kickoff en prosa legible — lo único que permite juzgar el prompt que recibirá el agente — y **la tanda entera**, con el problema de cada slice anotado en su propio bloque y un resumen de cuál rompería primero.

CANAL	CUENTA
stdout = el producto (plan, selección, motivo de bloqueo). stderr = diagnóstico (<code>aviso:</code> , <code>ATENCIÓN:</code> , abortos).	Imprime siempre qué cuenta de Claude ha elegido, por qué regla, y qué wrapper se teclea.

S9

• COORD.

Despachar: claim, worktree, semilla, lanzamiento verificado

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-next.mjs --repo "<owner/repo>"
```

Cinco actos, en orden, y cada uno con su reverso si falla:

- **Diagnóstico primero.** Colisión de orden, ambigüedad de `status:` , dependencias fuera de sección — impresos *antes* de seleccionar nada. Nunca bloquean por sí solos.
- **Selección.** Por orden dentro del epic, con las `merge-after` satisfechas (issue cerrado como *completed*), sin colisión de tokens `area:` / `touches:` y con hueco de `--cap` .
- **Claim en código,** no por prompt: `dispatch-check <issue> --repo <repo>` mueve `ready` → `in-progress` ANTES de crear el worktree.
- **Aislamiento.** `.worktrees/<n>` + rama `feat/<n>` , semilla de `.agent/SLICE.md` , y la regla de ignorado escrita en el `info/exclude` del directorio común — **con verificación del efecto:** si `git status` sigue viendo el fichero, el dispatch se aborta y el claim se revierte.
- **Lanzamiento con centinela.** `cmux --command` no ejecuta: *teclea*. Se teclea una línea corta que sourcea un launcher en disco (el kickoff viaja por fichero, donde ningún prompt puede morderlo), y ese launcher escribe un centinela con el `$PWD` real. Sin centinela no se dice «lanzado»: se reintenta el tecleo cada 2500 ms hasta agotar los 15 s de presupuesto.

LA REGLA QUE QUEDA

Si no se puede confirmar que el agente arrancó, el resultado no puede ser «lanzado» con exit 0. Y con el centinela ausente al agotarse la cota **tampoco se revierte el claim:** revertirlo con un agente que arranca tarde destruiría trabajo vivo, mientras que el residuo declarado con sus comandos exactos es recuperable.

S10

● HOOK

Hidratación y arranque verification-first

El hook `SessionStart` inyecta el estado en toda sesión nueva del worktree. La regla de precedencia es única para todos los lectores: **si existe `.agent/SLICE.md`, ése es el estado; si no, `.agent/STATE.md`**. La presencia del fichero es la señal de «estoy en un worktree de slice» — no hay variable de entorno ni detección de rutas. Y todo mensaje nombra el fichero que de verdad leyó, no una constante.

El primer acto del agente no es tocar código: **confirmar `pwd /rama`, `git log`, y `baseline verde` antes de nada.**

SI BLOCKED ESTÁ PUESTO

El hook lo anuncia destacado y declara el `next_action` SUSPENDIDO. Levantarlo es una decisión humana explícita.

HOOK STOP

Bloquea el cierre de turno si hay trabajo por encima de `last_commit` sin registrar. Un commit que solo toca el fichero de estado no cuenta.

S11

● AGENTE

Primer acto: el plan del slice, contra el código real

El plan del epic (`## Enfoque técnico`) es previo a la tabla. El **plan del slice** es posterior, y lo escribe el agente despachado al arrancar con `control-tower-loop:writing-plans` **usando el issue como spec**: sus AC en EARS, su `## Out of scope / Protected` y su `## Contexto del epic` son exactamente la entrada que `writing-plans` pide.

Se guarda en `docs/superpowers/plans/` y se commitea: **viaja en el PR**. Con el plan escrito, el rombo «¿hay plan?» de `subagent-driven-development` encuentra plan y el skill reengancha entero.

POR QUÉ AQUÍ

Se escribe contra el código real, en el momento correcto. Un plan escrito en el spec sería un plan escrito a ciegas.

S12

● AGENTE

Implementar: subagente por task, TDD, dos revisiones

`control-tower-loop:subagent-driven-development` con TDD: un subagente fresco por task, revisión entre tasks, iteración rápida. Los `Acepta` del issue son **1:1 con tests** — es lo que significa la cabecera `## Acceptance criteria` (EARS, `1:1 con tests`).

Lo que el agente **no** hace, dicho en el kickoff y no solo en los skills: no mergea el PR, no empieza el siguiente slice, y **no crea worktrees nuevos** — el aislamiento ya lo puso el dispatcher.

SI SE BLOQUEA

`blocked: {reason, unblock}` en `.agent/SLICE.md`, **nunca en prosa dentro de `next_action`**. El dispatcher lee ese campo y lo reporta con el motivo que el propio agente escribió.

S13

● AGENTE

Cerrar el slice: PR, release, parar

`finishing-a-development-branch` tiene un paso 0 nuevo: si existe `.agent/SLICE.md`, **no hay menú**. El camino es fijo: tests verdes → push + PR → comando literal de release → **PARAR**.

```
node <dispatch-check> <issue> --repo <owner/repo> --release
```

El PR lleva `Closes #N` **en el cuerpo** — no en el título, no en un comentario: GitHub solo interpreta las closing keywords en el cuerpo del PR y en los mensajes de commit. Y no es cosmético: es lo único que cierra el issue al mergear. Un PR mergeado con su issue abierto deja el slice reteniendo sus tokens para siempre, tapa el carril serializante, y ningún dependiente ve su `merge-after` satisfecho jamás.

--RELEASE SUELTA

El `--cap` : deja de contar como agente vivo.

PUERTA DE CONTAMINACIÓN

Se niega con exit 5 si la rama introduce `.agent/STATE.md` o `.agent/SLICE.md`, si la base no se puede determinar, o si la base resuelta es el mismo commit que HEAD.

--RELEASE NO SUELTA

Los tokens `area: / touches:`. Un `in-review` los retiene **hasta el merge**.

FASE 4 Cierre y cosecha humano + /ct-status

S14

● HUMANO

Puerta 3 — revisar, cerrar gates, decidir

Dos gates humanos, vocabulario cerrado: **visual** — un humano tiene que VER el cambio, con captura o vídeo del antes/después en el PR; **apply** — nada se aplica contra un entorno real hasta que un humano revise el plan o el dry-run.

El loop **escribe y enseña** los gates — label `gate:<token>`, sección `## Gates` del issue, línea explícita en el kickoff, campo `gates` del `SLICE.md` — pero **no impide mergear con uno sin cerrar**. El que los cierra es el humano.

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-status.mjs --repo "<owner/repo
```

`/ct-status` responde de una vez la pregunta que el coordinador se contestaba a mano: qué está en vuelo, qué ha entregado y qué es residuo. **No muta nada** — ni una escritura en todo el camino, con un test que lo comprueba mirando el argv real con el que se llamó a `gh`. Y ninguna lectura lleva `--limit`: todo pagina.

UNA REGLA QUE EL INFORME NO PUEDE APLICAR POR TI

Lo que compruebes antes de mergear tiene que poder detener el merge. Este comando imprime; no bloquea nada. Una comprobación cuyo resultado no detiene la acción siguiente es decoración, no comprobación.

S15

● HUMANO

Mergear — el único acto que desbloquea a los vecinos

El merge dispara la cadena entera: `Closes #N` cierra el issue como *completed* → los tokens `area:` / `touches:` quedan libres → el `merge-after` de cada dependiente pasa a estar satisfecho. Ni el `--release`, ni el `--reopen`, ni nada automático hace esto.

CONSECUENCIA PRÁCTICA

Un PR sin mergear frena a sus vecinos de área. Reabrir tampoco los desbloquea: lo único que los desbloquea es el merge.

TRAMPA CONOCIDA

Las closing keywords de **cualquier mensaje de commit** que llegue a la rama por defecto aplican, y las comillas no protegen. El dispatcher *avisa* cuando una dependencia consta cerrada por un commit suelto que no pertenece a ningún PR mergeado — pero no bloquea.

S16

● COORD.

Cosechar el residuo y anotar la evidencia

Todos los caminos del plugin que borran un worktree son de fallo; **ninguno cubre el éxito**. Observado en campo: PR mergeado, issue cerrado, y `.worktrees/451` + `feat/451` seguían en disco con su agente trece horas vivo. Con seis slices son seis checkouts, seis ramas muertas y seis agentes zombis — y el worktree de un slice terminado **bloquea el redespacho de ese número**.

```
git worktree remove .worktrees/<n> && git branch -d feat/<n>
```

Cada corrida de `/ct-next` cruza los issues mergeados contra el disco y emite `cosecha pendiente:` con los comandos exactos. **No borra nada**, y es deliberado: «mergeado» no es «nadie está tocando eso», y borrar un worktree

es irreversible. Por último, el `## Registro de cierre` del spec recoge la evidencia con la que se cerró cada gate — un gate cerrado sin evidencia es un gate no cerrado.

Y VUELTA AL S8

Con los tokens libres y el worktree cosechado, el siguiente slice del epic ya es despachable.

Las aristas de vuelta

Tres salidas de la senda feliz, todas deliberadamente manuales: nada automático las invoca.

SITUACIÓN	COMANDO	TRANSICIÓN	QUÉ COMPRUEBA
PR rechazado en revisión — se corrige encima de lo que hay	<code>dispatch-check <n> --reopen</code>	<code>in-review</code> → <code>in-progress</code>	Exige <code>status:in-review</code> exacto (exit 2 si no). Va a <code>in-progress</code> no a <code>ready</code> porque <code>ready</code> no retiene tokens y un vecino despacharía sobre una base que no contiene ese trabajo. No borra nada de disco: imprime lo que queda.
Abandonar el slice — empezar de cero, o romper un claim muerto	<code>dispatch-check <n> --requeue</code>	<code>in-progress</code> → <code>ready</code>	La única transición que suelta tokens: merge, así que comprueba antes de declarar: exige <code>in-progress</code> exacto y que no queden <code>.worktrees/<n> feat/<n></code> . Si no se pudo mirar, se niega: no se declara ausente lo que no ha visto.
Trabajo bloqueado — no se puede continuar	Campo <code>blocked</code> del <code>SLICE.md</code>	— (sin transición)	Ninguna transición loop lo saca de a claim rancio no lo (worktree y ramas existen), <code>--requeue</code> se niega por eso mismo, y <code>--rele</code> mentiría. El dispatcher lo lee y avisa. No arreglo automático es una decisión humana.

Un slice es una label

No hay base de datos. El estado de un slice son las labels de su issue de GitHub, y el timeline del issue registra cada transición con su timestamp. Eso hace que el estado sobreviva a un `/clear`, a un redespacho y a otra máquina — y que dos recursos distintos, el `--cap` y los tokens de área, se liberen en momentos distintos.

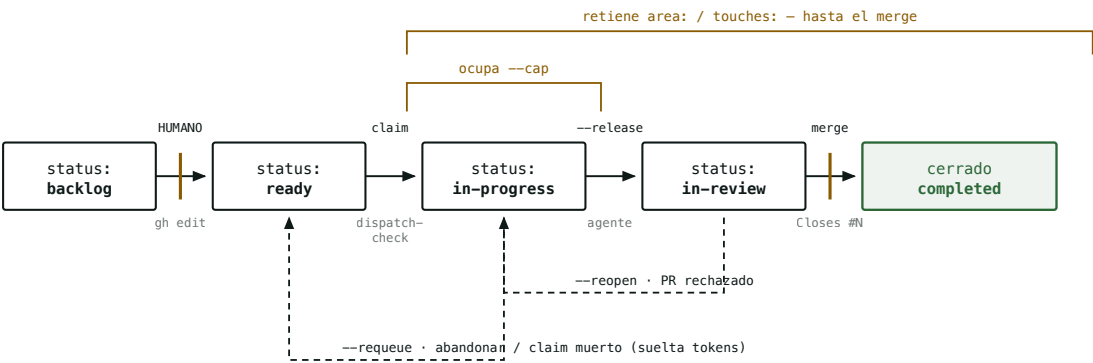


Fig. 2 — Los cinco estados y sus dos aristas de vuelta. Las barras verticales ámbar marcan las dos transiciones que sólo un humano ejecuta. Nótese la asimetría que explica la mayoría de los bloqueos reales: el `--release` libera el `--cap` pero **no** los tokens de área, así que un PR abierto sin mergear frena a sus vecinos aunque no haya ningún agente corriendo.

	STATUS: IN-PROGRESS	STATUS: IN-REVIEW
Ocupa <code>--cap</code>	sí (hay un agente corriendo)	no
Retiene <code>area: / touches:</code>	sí	sí, hasta el merge
Satisface un <code>merge-after</code>	no	no (hace falta cerrar como <i>completed</i>)
Detección de claim rancio	sí (se cruza con worktree / rama / sesión cmux)	no — no tener sesión ahí es lo normal

Once formatos, y qué viaja en cada uno

El principio que ordena todo el catálogo: **lo que escribas fuera de la tabla de slices y de ## Contexto del epic no llega al agente**. El agente que implementa un slice no recibe el spec — recibe un prompt de arranque y el cuerpo del issue. Una exigencia escrita en otra sección es invisible por muy contundente que esté redactada.

docs/superpowers/specs/YYYY-MM-DD-<tema>-design.md

COORDINADORA

El diseño validado en el brainstorming, aprobado sección a sección. No tiene plantilla rígida: cubre arquitectura, componentes, flujo de datos, manejo de errores y testing, con cada sección escalada a su complejidad — unas frases si es directo, hasta 200–300 palabras si tiene matiz.

ESCRIBE brainstorming , paso 6	LEE El humano, sección a sección, durante el diseño	VIDA Commiteado. Tras la congelación es historia : nadie lo edita
DESTINO Es el Handoff origen del execution spec		

docs/superpowers/specs/YYYY-MM-DD-<tema>-execution.md

COORDINADORA

El artefacto central del nivel epic. Un fichero por epic, trackeado, **DRAFT → CONGELADA** , archivado al cerrar el epic. Siete secciones más cabecera, y de todas ellas **sólo una viaja al agente**.

PLANTILLA _TEMPLATE-execution-spec.md v2	PARSEADO POR /ct-groom : la tabla, ## Contexto del epic , ## Hipótesis y los marcadores [NEEDS CLARIFICATION	PUERTA Sin Estado: CONGELADA no hay groom
---	---	--

FORMATO

<Epic> – Execution spec

****Handoff origen:**** `docs/superpowers/specs/YYYY-MM-DD-<tema>-design.md`
****Fecha de congelación:**** [la pone el OK del humano, no la sesión que escribe]
****Estado:**** DRAFT

Hipótesis del experimento ← OBLIGATORIA. groom: exit 2 si falta o está
****Apuesta:**** [qué creemos que pasará – formulado para poder ser FALSO]
****Cómo sabremos que falló:**** [la observación concreta que la tumba]
****Anti-scope – qué NO hace este epic:**** [lo único que impide que crezca]

Decisiones congeladas ← una por línea, con PROCEDENCIA
– ****D-1 · <título corto>:**** – <la decisión>. *(Procedencia: hablada – «cita litera
– ****D-2 · <título corto>:**** – <la decisión>. *(Procedencia: deducida de D-1.)*

Enfoque técnico ← el plan del EPIC. De aquí salen Área/Toca

Contexto del epic ← LO ÚNICO QUE VIAJA AL AGENTE
Los invariantes van DENTRO: version floors, reglas de copy, naming,
rutas que no se tocan. Una línea cada uno, con valores exactos.

Tabla de slices ← contrato v16, ver más abajo. UNA sola po

Decisiones aparcadas (BLOCKED) ← huecos no hablados. Vacía al congelar =
| ID | Fila | Qué falta decidir | Opciones vistas | Estado |

Registro de cierre (evidencia) ← se rellena al cerrar cada slice
| Slice | specReviewedSha | codeReviewedSha | uiScreenshot | Gate cerrado con |

§ Tabla de slices – el contrato con /ct-groom

● PARSEADO

Es la única parte de un spec que un programa parsea. El contrato vive en `AGENTS.md` del repo (bloque `ct-init:slices-contract`, versionado, lo mantiene `/ct-init`) y no se duplica en el spec. La tabla se localiza **por su cabecera de columnas** `Slice` + `Dep`, nunca por un número de sección.

CABECERA EXACTA, COPIABLE TAL CUAL

#	Slice	Tipo	Entrega	Dep	Acepta	Protegido	Área	Toca	Gate
1	walking skeleton	ui	pantalla vacía navegable	–	la ruta /hoy respon				
2	scoring		backend	endpoint POST /score	– consume getPillars()				

COLUMNA	OBLIGATORIA	A DÓNDE LLEGA
#	sí	Entero puro (1 , 2 ...). Orden del slice y target de Dep . Nunca S1 ni **1** : la fila entera se descarta. Único dentro de su milestone , no del repo.
Slice	sí	Título del issue (#N <Slice>). También es la primera línea del kickoff y el nombre del workspace de cmux — por eso corto y legible, no una frase.
Tipo	no	Label type:<valor> + qué addendum técnico recibe el agente. Reconocidos: ui , backend , infra , bugfix . Otro valor no aborta, avisa.
Entrega	no	Sección ## Descripción del issue. Ya no alimenta el título.
Dep	—	#N del orden de esta misma tabla, separadas por coma. Alimenta el grafo merge-after . En el issue va entre backticks a propósito: un #N desnudo lo autoenlazaría GitHub al issue N.
Acepta	no	Sección ## Acceptance criteria (EARS, 1:1 con tests) , uno por línea. La coma separa siempre — se escapa con \ , .
Protegido	no	Sección ## Out of scope / Protected . Texto libre de una pieza: no se trocea por comas.
Área / Toca	no	Labels area:<x> / touches:<y> : los tokens con los que el dispatcher detecta colisión y fuerza serialización. Sin estas columnas, esa maquinaria queda inerte para ese slice.
Gate	no	Vocabulario cerrado : visual , apply . Un valor desconocido aborta (a diferencia de Tipo). Renuncia con ! delante. Celda vacía = «no he declarado nada», no «renuncio a todo».

«Sin valor» se escribe — , — , — , — , — o celda vacía: todas las variantes de guion valen, porque un móvil con autocorrección sustituye una por otra sin que lo notes.

Las 4 reglas de celda

Van en la plantilla y no en el contrato porque **las juzga un humano al congelar, no un parser**.

- **1 · Clarificado = convergencia.** Una fila está lista cuando dos lecturas independientes convergen en el mismo desenlace. Si te la puedes imaginar de dos formas, no lo está: [NEEDS CLARIFICATION: ...] y a preguntar.
- **2 · Acepta = postcondiciones, no acciones.** Cada criterio afirma el *estado observable resultante* («el token caducado deja la sesión en login», no «se refresca el token»). EARS

desambigua; el test 1:1 lo denota. La incompletitud de los criterios es casi toda resultado infra-especificado, no acción olvidada.

- **3 · Gate = residuo de los Acepta** . Lo observable que *no* puede tener test 1:1 va al Gate — ahí es donde entra el humano. El default por Tipo es una heurística, no la regla.
- **4 · Dep declara interfaz**. *La más importante de las cuatro*. La Entrega de una fila con Dep nombra **qué consume del slice anterior** — la función, el endpoint, el token, el fichero. Un Dep que sólo dice #2 es un solape sin declarar, y el solape sin declarar es lo que hace que las reviews se rechacen.

Cuerpo del issue de GitHub – lo que el agente lee de verdad

● GENERADO

El orden de las secciones no es cosmético: **qué entrega → con qué contexto se interpreta → cómo se verifica → qué falta para poder mergearlo → qué queda fuera**. Una cabecera conocida se reconoce por **igualdad exacta**, no por prefijo: un `## Acceptance criteria` propuestos por QA (borrador) escrito por un humano nunca se confunde con la sección real.

FORMATO, EN ORDEN

```

> Slice `#1` del epic. Spec: [docs/.../spec-execution.md § Tabla de slices](https:

## Descripción                                     ← sólo si `Entrega` trae valor real
<texto de Entrega>

## Contexto del epic                               ← sólo si el spec trae texto. IDÉNTICO
<copiado literal del spec; --reconcile lo mantiene al día>

## Contexto heredado                               ← SIEMPRE, con placeholder. groom NO L
<la escribe la coordinadora cuando algo ya mergeado condiciona a ESTE slice>

## Acceptance criteria (EARS, 1:1 con tests)
- <criterio 1>
- <criterio 2>

## Dependencias                                     ← sólo si hay deps
*(cada `#N` de esta sección es el ORDEN del slice, NO un número de issue)*
- merge-after `#1`

## Gates                                             ← SIEMPRE, también cuando no hay ningun
<gates resueltos, o la afirmación de que no hay>

## Out of scope / Protected                         ← SIEMPRE
-  <Protegido>      | - (ninguno declarado)

<!-- ct-order:1 -->                                ← marcador greppable de orden. Book

```

TU ZONA, Y DÓNDE TERMINA

Todo lo que hay **entre** `## Contexto heredado` y `## Acceptance criteria` es tuyo: `--reconcile` no escribe ni un byte ahí, ni en las cabeceras que pegues ni en la prosa que escribas alrededor. Aguanta que pegues el cuerpo de otro issue entero, que es justo lo que hace falta («el slice #2 dejó montado esto»).

El final de tu zona lo marca `## Acceptance criteria`. Si esa cabecera **no aparece exactamente una vez**, no hay forma de saber dónde termina lo tuyo — y entonces `--reconcile` no escribe nada por detrás del contexto heredado, se rinde en voz alta y te pide que la restaures. De los dos errores posibles, el caro es el que borra texto escrito a mano.

Qué compara `/ct-groom` en un issue que ya existe, y qué no

Las labels no son metadatos decorativos: son el estado ejecutable del loop. Se crean **sólo si no existen ya** en el repo, y groom nombra por stderr las nuevas junto a las reutilizadas — que es la forma de detectar que un spec ha inventado un sinónimo (`area:hoy` frente a `area:today`) en vez de reutilizar el vocabulario, que es lo único que hace funcionar la detección de colisión.

PREFIJO	VALORES	QUIÉN LO MUEVE	QUÉ DECIDE
<code>status:</code>	<code>backlog</code> · <code>ready</code> · <code>in-progress</code> · <code>in-review</code> · <code>blocked</code>	groom crea en <code>backlog</code> ; humano promueve; <code>dispatch-check</code> reclama y libera	Despachabilidad, <code>--cap</code> , retención de tokens
<code>type:</code>	columna <code>Tipo</code> — reconocidos <code>ui</code> , <code>backend</code> , <code>infra</code> , <code>bugfix</code>	groom	Addendum técnico del kickoff + gate por defecto
<code>area:</code>	columna <code>Área</code>	groom	Token de colisión: dos slices con el mismo área no corren a la vez
<code>touches:</code>	columna <code>Toca</code>	groom	Igual, más el carril serializante global: <code>migration</code> , <code>ci</code> , <code>pbxproj</code>
<code>gate:</code>	<code>visual</code> · <code>apply</code> · <code>none</code>	groom (de <code>Gate</code> , o derivado de <code>Tipo</code>)	El gate humano que sobrevive a un redespacho, a un <code>--reopen</code> y a un <code>/clear</code>

`gate:none` existe porque **el silencio no puede significar dos cosas distintas**: sin esa label, «este slice no exige ningún gate» y «este issue es anterior a los gates» serían indistinguibles. Un issue groomeado antes de que existieran cae al `Tipo` , así que no pierde el gate que ya tenía.

No es un fichero del repo: se renderiza en cada dispatch y se escribe en un launcher temporal que cmux sourcea. Once líneas, y el orden importa — **lo que se enumera se lee, y**

lo que se deja a «ya lo verá» compite con el resto del cuerpo del issue. Las secciones de prosa arbitraria se *nombran* en vez de interpolarse, porque esto se teclea entero en un pty.

ESTRUCTURA

- 1. Estás implementando UN slice (<Slice>) del repo <owner/repo>, issue #N (orden
- 2. Es human-gated: NO mergees el PR, NO empieces el siguiente slice, y NO crees worktrees nuevos – ya estás en el que te preparó el dispatcher.
- 3. Arranque verification-first: confirma pwd/rama, git log, y baseline verde ANTES de tocar nada.
- 4. Hidrátate de .agent/SLICE.md y del issue; los criterios de aceptación son <AC interpolados>. Lee además "## Out of scope / Protected": eso NO se toca.
- 5. Lee también "## Contexto del epic" y "## Contexto heredado". Si alguna está vacía o no aparece, no hay nada que heredar – no lo busques fuera del issue.
- 6. Primer acto, con el baseline verde: escribe el plan del slice con control-tower-loop:writing-plans usando el issue como spec. Guárdalo en docs/superpowers/plans/ y commitéalo: viaja en el PR.
- 7. Con el plan escrito, sigue control-tower-loop:subagent-driven-development con
- 8. <addendum técnico según Tipo: ui | backend | infra | bugfix>
- 9. <líneas de GATE – qué tiene que ver o revisar un humano antes del merge>
- 10. Si el trabajo queda BLOQUEADO, márcalo en .agent/SLICE.md como `blocked: {reason, unblock}` – NO en prosa dentro de next_action.
- 11. Al acabar: commit refs al issue, actualiza .agent/SLICE.md, abre el PR contr <base> con `Closes #N` en el CUERPO del PR (no en el título, no en un comentario), libera el claim con `node <dispatch-check> N --repo <r> --release` deja el estado mergeable y PARA.
+ el POR QUÉ de ese Closes, aparte: es lo ÚNICO que cierra el issue al mergea

.worktrees/<n>/ .agent/SLICE.md

● AGENTE DEL SLICE

El estado del slice. **Ignorado por git, nunca commiteable** — y la garantía no se confía a la documentación: /ct-next escribe la regla en el info/exclude del directorio común (una escritura cubre coordinadora y todos los worktrees) y **después verifica el efecto**. La presencia de este fichero es la señal de «estoy en un worktree de slice».

SIEMBRA /ct-next al despachar	ESCRIBE El agente del slice, turno a turno	LEE Los hooks SessionStart y Stop, y /ct-next (para detectar bloqueos)
GIT ignorado · --release sale con exit 5 si la rama lo introduce		

SEMILLA, TAL CUAL SE ESCRIBE

```
---
task: "<Slice>"
role: "slice-agent (sesión DESPACHADA por /ct-next): implementas ESTE slice y PA
    No groomeas, no mergeas, no despachas el siguiente."
gates: "visual – GATES HUMANOS pendientes: los cierra quien revisa el PR, NO tú.
    # o: "ninguno (este slice no exige ningún gate humano antes de mergear)"
status: not_started
branch: "feat/<n>"
base: main
last_commit: "<SHA de la base>" # un SHA, no un nombre de rama: sin él el hook
github_issue: <n>
you_are_here: "worktree fresco para el issue #N de GitHub (orden #M)"
next_action: "hidrátate del issue y empieza por el primer AC (<AC-1>)"
blocked: null # explícito, no omitido
verify: "" # la comprobación PENDIENTE, nunca un hecho ya
tasks: []
---
## Current State
(slice recién despachado, sin trabajo aún)
```

`role` y `gates` son **campos** y no frases dentro de un prompt por la misma razón que `blocked`: lo que tiene que sobrevivir a una re-hidratación es un campo. El kickoff se pierde con el contexto de su sesión; este fichero lo inyecta el hook en toda sesión nueva del worktree. Ningún código del plugin decide nada con `role` ni con `gates` — el gate ejecutable son las labels.

.agent/STATE.md (checkout principal)

● COORDINADORA

El otro fichero de estado, y el que **no** habla de ningún slice. Está trackeado y el dispatcher no lo toca: en el worktree de un slice se queda a cero diff. Mismos campos que el `SLICE.md`, con un `role` opuesto.

```
role: "coordinador (checkout principal): groomeas, despachas con /ct-next, revis
    mergeas. NO implementas slices aquí – eso pasa en .worktrees/<n>."
next_action: "(sin slice asignado)" # /ct-init lo deja así a propósito
blocked: null
```

POR QUÉ `/CT-INIT` NO RELLENA `NEXT_ACTION`

Es un scaffolder, no un planificador. Si apunta a un pendiente encontrado por el repo, el hook de `SessionStart` hidratará **todas** las sesiones futuras de ese repo creyendo que eso es lo siguiente — aunque no tenga nada que ver con lo que se vaya a hacer.

docs/superpowers/plans/YYYY-MM-DD-<feature>.md

• AGENTE DEL SLICE

El plan del slice, escrito contra el código real y commiteado en el PR. Se escribe asumiendo un ingeniero **con cero contexto del codebase**: rutas exactas siempre, código completo en cada paso, comandos exactos con su salida esperada.

CABECERA OBLIGATORIA + ESTRUCTURA DE UNA TASK

<Feature> Implementation Plan

> ****For agentic workers:**** REQUIRED SUB-SKILL: use control-tower-loop:subagent-d
> development (recommended) or control-tower-loop:executing-plans. Steps use `-

****Goal:**** <una frase>

****Architecture:**** <2-3 frases>

****Tech Stack:**** <tecnologías clave>

Global Constraints

[los invariantes del spec – version floors, límites de dependencias, reglas de naming y copy – una línea cada uno, con los valores exactos copiados literalmente. Los requisitos de CADA task los incluyen implícitamente.]

Task N: <Component>

****Files:****

- Create: `exact/path/to/file.ts`
- Modify: `exact/path/to/existing.ts:123-145`
- Test: `tests/exact/path/to/test.ts`

****Interfaces:****

- Consumes: [qué usa de tasks anteriores – firmas exactas]
- Produces: [de qué dependen las siguientes – nombres, tipos de parámetro y retorno. El implementador de una task sólo ve la suya: este bloque es cómo aprende los nombres que usan sus vecinas.]

- [] ****Step 1: Write the failing test**** ← con el código del test, no una descripción
- [] ****Step 2: Run test to verify it fails**** ← con el comando y el fallo esperado
- [] ****Step 3: Write minimal implementation****
- [] ****Step 4: Run test to verify it passes****
- [] ****Step 5: Commit****

FALLOS DE PLAN – NUNCA SE ESCRIBEN

«TBD», «implement later», «add appropriate error handling», «handle edge cases», «write tests for the above» sin el código, «similar to Task N» (se repite el código: el ingeniero puede leer las tasks en otro orden), y referencias a tipos o funciones que ninguna task define.

EL PR

● AGENTE

● HUMANO

El paquete de entrega del slice. Contiene el código, los tests, **el plan commiteado** y — si el slice lleva gate `visual` — la captura del antes/después. Se abre contra la base del despacho y el agente **para**.

RAMA	OBLIGATORIO EN EL CUERPO	QUIÉN MERGEA
feat/<n> — determinista, es lo que permite cruzarla con el issue	Closes #N — no en el título, no en un comentario	El humano. Siempre.

.agent/conventions-ack.md

● HUMANO

El acuse de recibo que hace que un aviso correcto **deje de avisar sin borrar documentación correcta**. Antes de que existiera, la única forma de callar un aviso era borrar el texto que lo disparaba.

```
claim: 2026-07-28 — manda el claim del plugin; scripts/dispatch-check.sh se queda para trabajo a mano fuera del loop
residuo-status: 2026-07-28 — #101, #102 <por qué se quedan así>
```

Silencia **esa** señal y sólo esa; las demás siguen avisando, y en las corridas siguientes queda una nota de una línea diciendo que está silenciada y desde cuándo — «decidido no mirar esto» y «aquí no hay nada» no son lo mismo. Hacen falta las tres cosas (señal, fecha y motivo): una línea que no parsee se reporta y no silencia nada. Y un acuse **sin números** no silencia nada: no hay comodín para «todos», que devolvería el mismo silencio estático que el mecanismo corrige.

Los códigos de salida, que son la interfaz real

Los cuatro ejecutables comparten convención de canal — **stdout es el producto, stderr es el diagnóstico** — y una gramática de tres estados: hecho, no se pudo comprobar, queda algo pendiente. Si `/ct-next` corre dentro de un `/loop`, esta tabla es lo que decide si toca seguir, parar o reintentar.

`/ct-groom`

EXIT	SIGNIFICA	QUÉ HACER
0	Sin divergencia real, o <code>--reconcile</code> la resolvió del todo	Nada
1	La corrida se paró en seco: una de las dos puertas de alcance del epic, un fallo de lectura de <code>gh</code> , un fallo de escritura a mitad, o algo de <code>--project</code> que no se pudo dar por bueno	Leer el mensaje: dice <code>no se ha creado ni modificado nada</code> cuando así es
2	La tabla es inválida, o el spec no pasa la puerta de congelación (<code>[NEEDS CLARIFICATION</code> pendiente, <code>## Hipótesis</code> ausente o vacía)	Arreglar el spec. Todos los fallos salen juntos, no de uno en uno
3	Queda algo real sin reconciliar: divergencia de título/enlace/labels/AC/deps, una sección duplicada, o un issue huérfano	Revisar. Ojo con <code>groom --dry-run && groom</code> : el 3 del dry-run corta la cadena justo cuando hay algo que aplicar

EXIT	SIGNIFICA	QUÉ HACER
0	Progreso (algo se lanzó, total o parcialmente), o no había nada seleccionable y ya se explicó por qué	Seguir con normalidad
1	Algo se rompió o quedó a medias que un humano tiene que resolver: un issue huérfano en <code>in-progress</code> , una precondición sin cumplir, o al menos un slice lanzado sin verificar	Parar. Ante un lanzamiento sin verificar, mirar la sesión de cmux ANTES de borrar nada: revertir el claim con un agente vivo es peor que dejarlo
2	Error de uso o de configuración estática: flags mal puestos, <code>ACCOUNT_MAP</code> malformado, nombre de binario que no es un comando a secas	Corregir la invocación o el mapa
3	Se seleccionó tanda y terminó con cero lanzamientos sin que nada quedara a medias: todos los candidatos se saltaron al reclamar. Ningún claim escrito, ninguna rama ni worktree creados	Reintentar más tarde. No es una señal de alarma, y ahora el mensaje puede afirmarlo con verdad
130 / 143	SIGINT / SIGTERM. Una interrupción siempre se acusa , aunque llegue con la tanda ya terminada	Si llegó entre el claim y el worktree, el claim ya se revirtió automáticamente

/ct-status y dispatch-check

COMANDO	EXIT	SIGNIFICA
ct-status	0	Nada que revisar: nada en vuelo sin señales de vida, ningún residuo, ninguna lectura a medias
	3	Hay algo que revisar: residuo, un claim sin proceso vivo, labels huérfanas, entregas sin cosechar
	1	No se pudo comprobar. La precedencia es <code>1 > 3 > 0</code> , nunca al revés: una lectura incompleta no es un loop en reposo, y un <code>0</code> sobre datos truncados es exactamente el fallo que este comando viene a eliminar
dispatch-check	1	Colisión detectada a tiempo, o carrera perdida con revert limpio. Resultado normal del protocolo: ese slice se salta y se sigue con la tanda
	3	Fallo de infraestructura puntual — se trata igual que una colisión, pero el mensaje deja explícito que es un <code>gh</code> que falló
	4	Issue huérfano: carrera perdida o fallo de readback, y el revert posterior también falló. El issue queda bloqueado en <code>in-progress</code> sin nadie trabajándolo. Esto para la tanda entera
	5	<code>--release</code> se niega: la rama introduce un fichero de estado, la base no se puede determinar, o la base resuelta es el mismo commit que HEAD

Lo que el loop no garantiza

Escrito aquí porque un límite conocido y dicho es operable, y uno implícito no.

EL CLAIM NO ES ATÓMICO

El lock con el que `dispatch-check` reclama un issue son labels de GitHub, **sin compare-and-swap**. Está reproducido y verificado —no sospechado— que dos dispatchers lanzados casi a la vez pueden reclamar el mismo token compartido y arrancar los dos. No hay ninguna espera ni reintento que cierre este hueco hoy.

La mitigación es operativa: no lances dos `/ct-next` al mismo tiempo contra el mismo repo. El plan es migrar el claim a una primitiva atómica real vía refs de git; hasta entonces, ésta es la única garantía que existe.

NADA VIGILA LOS CLAIMS ENTRE INVOCACIONES

El claim es una label, sin heartbeat: sólo se entera quien esté corriendo `/ct-next` o `/ct-status` en ese momento. Y la evidencia de vida es **local a esta máquina**, así que nunca se afirma «abandonado», sólo «aquí no hay ni rastro». Si el loop se despacha desde más de un sitio, «sin señal de vida» significa «no lo está trabajando nadie *aquí*».

- **Los gates se escriben y se enseñan; no se imponen.** El loop no impide mergear con un gate sin cerrar. Es el humano.
- **No hay transacción en el groom.** Superada la validación, un fallo de red deja creado lo anterior. No hay rollback y no se finge que lo haya: de un abort a mitad se sale volviendo a correr.
- **El enlace al spec apunta a la rama por defecto.** Si el fichero se mueve o se renombra, el enlace de los issues ya creados muere. Se detecta en la siguiente corrida, pero no se corrige sin `--reconcile`. **Groomea después de empujar el spec.**
- **Un `merge-after #N` escrito fuera de su sección no gatea nada** — se avisa, pero se despacha igual. El dominio de detección y el de aplicación son, deliberadamente, el mismo: sólo la sección reconocida.
- **`dist/` es lo que se ejecuta, no `hooks/`.** Todo cambio en los hooks tiene que llevar un `dist/` reconstruido en el mismo commit, o el repo distribuye el hook viejo mientras la fuente dice otra cosa. Y la suite se queda en verde, porque `npm test` construye primero: prueba un `dist/` recién hecho mientras el commiteado se pudre.
- **La medida está pre-registrada, no cosechada.** El timeline de GitHub ya registra cada transición de label con su timestamp — media instrumentación existe sin recoger. El criterio de muerte sigue vigente: *si el loop cuesta más intervención humana de la que ahorra, se dice y se para*.

Compuesto a partir del código y la documentación del plugin `control-tower-loop` en `/Users/jpereag/Documents/control-tower-plugin` : los cuatro `commands/*.md` , los skills forkados de `skills/` , los generadores de `scripts/groom.js` y `scripts/kickoff.js` , el contrato v16 de `scripts/ct-init.sh` , y la plantilla v2 del execution spec. Cada formato de esta página sale de la fuente que lo emite, no de una descripción de segunda mano.